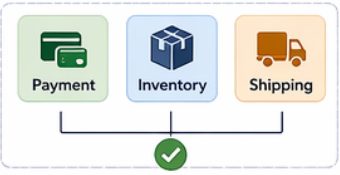
DAY 16 Two-Phase Commit, Distributed Transactions, and Recovery

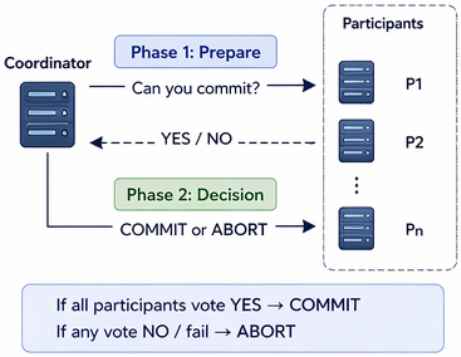
Coordinating multiple services so they either all commit or all abort.



WHY DISTRIBUTED TRANSACTIONS?

- A business action may touch multiple services.
- Independent local commits can leave partial state if one step fails.
- A distributed transaction protocol coordinates these services under one final decision.

TWO-PHASE COMMIT OVERVIEW



If all participants vote YES → COMMIT
If any vote NO / fail → ABORT

TWO PHASES

1 PREPARE PHASE

- Coordinator asks all participants if they can commit.
- Participants validate, reserve resources, and respond YES or NO.

2 DECISION PHASE

- If all vote YES → Coordinator sends COMMIT to all.
- If any vote NO or timeout → Coordinator sends ABORT to all.

ROLES & RESPONSIBILITIES

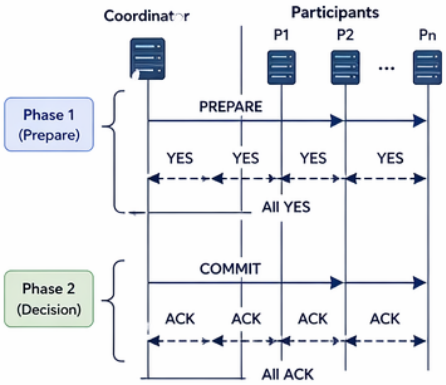
COORDINATOR

- Starts the transaction.
- Sends prepare requests.
- Collects votes.
- Logs final decision (COMMIT / ABORT).
- Sends decision to all participants.
- Handles recovery and reconciliation.

PARTICIPANT

- Receives prepare request.
- Validates and reserves resources.
- Responds YES or NO.
- On decision: commits or aborts locally.
- Can recover and request decision if it was missed.

2PC MESSAGE FLOW



DECISION OUTCOMES

COMMIT

- All participants voted YES.
- Coordinator logs COMMIT.
- All participants commit and release resources.

ABORT

- Any participant voted NO or failed to respond.
- Coordinator logs ABORT.
- All participants abort and release resources.

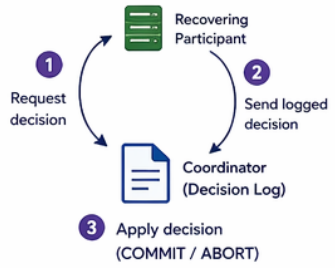
FAILURES & UNCERTAIN STATES

Coordinator fails before decision
Participants stay in PREPARED (uncertain) state. They wait for decision after recovery.

Participant misses decision
Participant remains uncertain. It must obtain decision from coordinator when it recovers.

Network partition
Minority side should not make progress. Decision requires coordinator and majority to be healthy.

RECOVERY & RECONCILIATION



TRADE-OFFS

- 2PC provides atomicity across services but adds latency.
- Coordinator failure can cause blocking (participants wait).
- Strong consistency vs availability trade-off.
- Many systems prefer Saga when high availability is more important than strict atomicity.

EXAMPLE FLOW

1. Client starts transaction
2. Coordinator sends PREPARE to all participants
3. Participants validate and respond YES / NO
4. Coordinator logs decision and sends COMMIT or ABORT
5. Participants commit or abort and send acknowledgement
6. Transaction completed

KEY TAKEAWAYS

- 2PC ensures all-or-nothing outcome across participants.
- Prepare phase reserves resources; decision phase finalizes.
- Decision logging is critical for recovery correctness.
- Uncertain states are normal and must be handled.
- 2PC and Saga are complementary choices.

QUICK REFERENCE

Phase 1 (Prepare)	Ask participants to prepare and vote YES / NO
Phase 2 (Decision)	Send COMMIT if all YES, otherwise ABORT
Commit Condition	All participants voted YES
Abort Condition	Any NO vote, timeout, or failure
Durability	Coordinator must persist final decision
Recovery	Reconcile uncertain participants using log

PARTICIPANT STATES

INITIAL	No transaction yet.
PREPARED	Voted YES, waiting for decision.
COMMITTED	Transaction committed.
ABORTED	Transaction aborted.

System Design - Day 16

Two-Phase Commit, Distributed Transactions, and Recovery

Prepared from Day-16 notes and sample implementation

Overview from README.md

Day 16: Two-Phase Commit, Distributed Transactions, and Recovery

Day 16 focuses on how multiple services coordinate one logical transaction so they either all commit or all abort.

It covers:
- distributed transaction fundamentals
- two-phase commit (`prepare` and `commit/abort`)
- coordinator and participant roles
- commit durability and decision logging
- failure handling and uncertain participant states
- recovery and decision reconciliation

In short, Day 16 is about preserving cross-service consistency when one operation spans multiple systems.

Core ideas

Why distributed transactions matter
- A business action may touch multiple services (for example, payment, inventory, shipping).
- Independent local commits can leave partial state if one step fails.
- A distributed transaction protocol coordinates these services under one final decision.

Two-phase commit phases
- Phase 1 (`prepare`): coordinator asks participants if they can commit.
- Participants reserve resources and respond `YES` or `NO`.
- Phase 2 (`decision`): if all voted `YES`, coordinator sends `COMMIT`; otherwise `ABORT`.

Coordinator and participant responsibilities
- Coordinator drives the protocol, collects votes, and records final decision.
- Participants keep temporary/pending state during prepare.
- Participants finalize local state only after receiving the coordinator decision.

Failure and recovery behavior
- If a participant misses the final decision (for example, temporary outage), it can remain uncertain.
- Coordinator must persist decision so it can be replayed later.
- Recovery path reconciles uncertain participants with the recorded decision.

Trade-offs
- 2PC improves atomicity across services but adds latency and coordination overhead.
- Blocking can occur if coordinator fails before participants learn final decision.
- Many large systems prefer Saga patterns when high availability is prioritized over strict atomicity.

Day-16 sample: Two-Phase Commit Coordinator

This repository includes a small 2PC sample in both Python and Java.

Functional requirements
- Execute a transaction across multiple named participants
- Collect prepare votes before finalizing
- Commit only when all participants vote `YES`

- Abort when any participant votes `NO` or is unavailable
- Persist transaction decision for later reconciliation
- Recover a participant that missed the final decision

High-level components
- `Participant`: handles prepare/finalize and keeps pending/committed state
- `Coordinator`: runs phase-1 and phase-2 logic
- `Decision log`: records `COMMIT` or `ABORT` per transaction
- `Reconcile path`: applies logged decisions to recovering participants

Data flow
1. Client starts transaction with operations per participant
2. Coordinator sends prepare request to each participant
3. Participants vote and hold pending state if prepared
4. Coordinator logs final decision (`COMMIT` or `ABORT`)
5. Coordinator broadcasts decision to prepared participants
6. Recovering participant can request and apply logged decision later

Project structure

```
Day-16/
README.md
two-phase-commit-sample/
python/
two_phase_commit.py
demo.py
java/
TwoPhaseCommitCoordinator.java
Main.java
```

Run the sample

Python

```
cd two-phase-commit-sample\python
python demo.py
```

Java

```
cd two-phase-commit-sample\java
javac Main.java TwoPhaseCommitCoordinator.java
java Main
```

Interview takeaways
- 2PC gives all-or-nothing behavior across participants at the cost of coordination overhead
- Decision durability at coordinator is key for recovery correctness
- Participant uncertainty after failures is a core operational concern
- Blocking characteristics of 2PC are important when discussing availability trade-offs
- 2PC and Saga are complementary choices based on consistency requirements

Next improvements
- Add coordinator crash and restart simulation with replayed decision log
- Add participant timeouts and retransmission behavior
- Add idempotent decision handling for repeated commit/abort messages
- Add Saga alternative flow for comparison in the same sample
- Add metrics for prepared duration and transaction completion latency

Reference Implementation

Python and Java sample source files

File: two-phase-commit-sample/python/demo.py

```python
from two_phase_commit import TwoPhaseCommitCoordinator


def print_tx_state(coordinator: TwoPhaseCommitCoordinator, tx_id: str) -> None:
    for line in coordinator.states_for_tx(tx_id):
        print("  " + line)


def main() -> None:
    coordinator = TwoPhaseCommitCoordinator(["inventory", "payment", "shipping"])

    tx_id = "tx-100"
    operations = {
        "inventory": "reserve sku=book-1 qty=1",
        "payment": "charge user=42 amount=499",
        "shipping": "create shipment user=42",
    }
    decision = coordinator.execute(tx_id, operations)
    print(f"{tx_id} decision: {decision.value}")
    print_tx_state(coordinator, tx_id)

    tx_id = "tx-101"
    coordinator.participant("payment").set_reject_for_tx(tx_id, True)
    decision = coordinator.execute(tx_id, operations)
    print(f"\n{tx_id} decision (payment votes NO): {decision.value}")
    print_tx_state(coordinator, tx_id)
    coordinator.participant("payment").set_reject_for_tx(tx_id, False)

    tx_id = "tx-102"
    coordinator.participant("shipping").set_available(True)
    decision = coordinator.execute(tx_id, operations, drop_before_decision=["shipping"])
    print(f"\n{tx_id} decision (shipping misses decision broadcast): {decision.value}")
    print_tx_state(coordinator, tx_id)

    coordinator.participant("shipping").set_available(True)
    coordinator.reconcile_participant("shipping", tx_id)
    print(f"\n{tx_id} after shipping recovery + reconcile:")
    print_tx_state(coordinator, tx_id)


if __name__ == "__main__":
    main()
```

File: two-phase-commit-sample/python/two_phase_commit.py

```python
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum


class Decision(Enum):
    COMMIT = "COMMIT"
    ABORT = "ABORT"


@dataclass(frozen=True)
class CommittedOperation:
    tx_id: str
    operation: str


class Participant:
    def __init__(self, name: str) -> None:
        self.name = name
        self.available = True
        self._reject_tx_ids: set[str] = set()
        self._pending: dict[str, str] = {}
        self._committed: list[CommittedOperation] = []

    def set_available(self, available: bool) -> None:
        self.available = available

    def set_reject_for_tx(self, tx_id: str, reject: bool) -> None:
        if reject:
            self._reject_tx_ids.add(tx_id)
        else:
            self._reject_tx_ids.discard(tx_id)

    def prepare(self, tx_id: str, operation: str) -> bool:
        if not self.available:
            return False
        if tx_id in self._reject_tx_ids:
            return False
        self._pending[tx_id] = operation
        return True

    def finalize(self, tx_id: str, decision: Decision) -> None:
        if not self.available:
            return
        operation = self._pending.pop(tx_id, None)
        if operation is None:
            return
        if decision == Decision.COMMIT:
            self._committed.append(CommittedOperation(tx_id=tx_id, operation=operation))

    def has_pending(self, tx_id: str) -> bool:
        return tx_id in self._pending

    def has_committed(self, tx_id: str) -> bool:
        return any(item.tx_id == tx_id for item in self._committed)


class TwoPhaseCommitCoordinator:
    def __init__(self, participant_names: list[str]) -> None:
        if not participant_names:
            raise ValueError("participant_names must be non-empty")
        if len(set(participant_names)) != len(participant_names):
            raise ValueError("participant names must be unique")

        self._participants = {name: Participant(name) for name in participant_names}
        self._decision_log: dict[str, Decision] = {}

    def participant(self, name: str) -> Participant:
        participant = self._participants.get(name)
        if participant is None:
            raise ValueError(f"unknown participant: {name}")
```

```
        return participant

    def execute(
        self,
        tx_id: str,
        operations_by_participant: dict[str, str],
        drop_before_decision: list[str] | None = None,
    ) -> Decision:
        if not tx_id:
            raise ValueError("tx_id must be non-empty")
        if not operations_by_participant:
            raise ValueError("operations_by_participant must be non-empty")

        prepared_participants: list[str] = []
        all_yes = True

        for participant_name, operation in operations_by_participant.items():
            participant = self.participant(participant_name)
            voted_yes = participant.prepare(tx_id, operation)
            if voted_yes:
                prepared_participants.append(participant_name)
            else:
                all_yes = False

        decision = Decision.COMMIT if all_yes else Decision.ABORT
        self._decision_log[tx_id] = decision

        for participant_name in drop_before_decision or []:
            self.participant(participant_name).set_available(False)

        for participant_name in prepared_participants:
            self.participant(participant_name).finalize(tx_id, decision)

        return decision

    def reconcile_participant(self, participant_name: str, tx_id: str) -> None:
        decision = self._decision_log.get(tx_id)
        if decision is None:
            raise ValueError(f"no decision found for tx_id={tx_id}")
        self.participant(participant_name).finalize(tx_id, decision)

    def decision_for(self, tx_id: str) -> Decision | None:
        return self._decision_log.get(tx_id)

    def states_for_tx(self, tx_id: str) -> list[str]:
        lines: list[str] = []
        for name, participant in self._participants.items():
            lines.append(
                f"{name}: available={participant.available}, "
                f"pending={participant.has_pending(tx_id)}, "
                f"committed={participant.has_committed(tx_id)}"
            )
        return lines
```

File: two-phase-commit-sample/java/Main.java

```java
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        TwoPhaseCommitCoordinator coordinator =
            new TwoPhaseCommitCoordinator(List.of("inventory", "payment", "shipping"));

        Map<String, String> operations = new LinkedHashMap<>();
        operations.put("inventory", "reserve sku=book-1 qty=1");
        operations.put("payment", "charge user=42 amount=499");
        operations.put("shipping", "create shipment user=42");

        String txId = "tx-100";
        TwoPhaseCommitCoordinator.Decision decision = coordinator.execute(txId, operations);
        System.out.println(txId + " decision: " + decision);
        printTxState(coordinator, txId);

        txId = "tx-101";
        coordinator.participant("payment").setRejectForTx(txId, true);
        decision = coordinator.execute(txId, operations);
        System.out.println("\n" + txId + " decision (payment votes NO): " + decision);
        printTxState(coordinator, txId);
        coordinator.participant("payment").setRejectForTx(txId, false);

        txId = "tx-102";
        coordinator.participant("shipping").setAvailable(true);
        decision = coordinator.execute(txId, operations, List.of("shipping"));
        System.out.println("\n" + txId + " decision (shipping misses decision broadcast): " + decision);
        printTxState(coordinator, txId);

        coordinator.participant("shipping").setAvailable(true);
        coordinator.reconcileParticipant("shipping", txId);
        System.out.println("\n" + txId + " after shipping recovery + reconcile:");
        printTxState(coordinator, txId);
    }

    private static void printTxState(TwoPhaseCommitCoordinator coordinator, String txId) {
        for (String line : coordinator.statesForTx(txId)) {
            System.out.println("  " + line);
        }
    }
}
```

File: two-phase-commit-sample/java/TwoPhaseCommitCoordinator.java

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;

public class TwoPhaseCommitCoordinator {
    public enum Decision {
        COMMIT,
        ABORT
    }

    private static final class CommittedOperation {
        private final String txId;
        private final String operation;

        private CommittedOperation(String txId, String operation) {
            this.txId = txId;
            this.operation = operation;
        }
    }

    public static final class Participant {
        private final String name;
        private boolean available = true;
        private final Set<String> rejectTxIds = new HashSet<>();
        private final Map<String, String> pending = new LinkedHashMap<>();
        private final List<CommittedOperation> committed = new ArrayList<>();

        private Participant(String name) {
            this.name = name;
        }

        public void setAvailable(boolean available) {
            this.available = available;
        }

        public void setRejectForTx(String txId, boolean reject) {
            if (reject) {
                rejectTxIds.add(txId);
            } else {
                rejectTxIds.remove(txId);
            }
        }

        private boolean prepare(String txId, String operation) {
            if (!available) {
                return false;
            }
            if (rejectTxIds.contains(txId)) {
                return false;
            }
            pending.put(txId, operation);
            return true;
        }

        private void finalizeDecision(String txId, Decision decision) {
            if (!available) {
                return;
            }
            String operation = pending.remove(txId);
            if (operation == null) {
                return;
            }
            if (decision == Decision.COMMIT) {
                committed.add(new CommittedOperation(txId, operation));
            }
        }

        private boolean hasPending(String txId) {
            return pending.containsKey(txId);
```

```
    }

    private boolean hasCommitted(String txId) {
        for (CommittedOperation operation : committed) {
            if (operation.txId.equals(txId)) {
                return true;
            }
        }
        return false;
    }
}

private final Map<String, Participant> participants;
private final Map<String, Decision> decisionLog;

public TwoPhaseCommitCoordinator(List<String> participantNames) {
    if (participantNames == null || participantNames.isEmpty()) {
        throw new IllegalArgumentException("participantNames must be non-empty");
    }

    participants = new LinkedHashMap<>();
    for (String name : participantNames) {
        if (participants.containsKey(name)) {
            throw new IllegalArgumentException("participant names must be unique");
        }
        participants.put(name, new Participant(name));
    }
    decisionLog = new LinkedHashMap<>();
}

public Participant participant(String name) {
    Participant participant = participants.get(name);
    if (participant == null) {
        throw new IllegalArgumentException("unknown participant: " + name);
    }
    return participant;
}

public Decision execute(
    String txId,
    Map<String, String> operationsByParticipant,
    List<String> dropBeforeDecision
) {
    if (txId == null || txId.isBlank()) {
        throw new IllegalArgumentException("txId must be non-empty");
    }
    if (operationsByParticipant == null || operationsByParticipant.isEmpty()) {
        throw new IllegalArgumentException("operationsByParticipant must be non-empty");
    }

    List<String> preparedParticipants = new ArrayList<>();
    boolean allYes = true;

    for (Map.Entry<String, String> entry : operationsByParticipant.entrySet()) {
        Participant participant = participant(entry.getKey());
        boolean votedYes = participant.prepare(txId, entry.getValue());
        if (votedYes) {
            preparedParticipants.add(entry.getKey());
        } else {
            allYes = false;
        }
    }

    Decision decision = allYes ? Decision.COMMIT : Decision.ABORT;
    decisionLog.put(txId, decision);

    if (dropBeforeDecision != null) {
        for (String name : dropBeforeDecision) {
            participant(name).setAvailable(false);
        }
    }

    for (String participantName : preparedParticipants) {
```

```
            participant(participantName).finalizeDecision(txId, decision);
        }

        return decision;
    }

    public Decision execute(String txId, Map<String, String> operationsByParticipant) {
        return execute(txId, operationsByParticipant, List.of());
    }

    public void reconcileParticipant(String participantName, String txId) {
        Decision decision = decisionLog.get(txId);
        if (decision == null) {
            throw new IllegalArgumentException("no decision found for txId=" + txId);
        }
        participant(participantName).finalizeDecision(txId, decision);
    }

    public Decision decisionFor(String txId) {
        return decisionLog.get(txId);
    }

    public List<String> statesForTx(String txId) {
        List<String> lines = new ArrayList<>();
        for (Participant participant : participants.values()) {
            lines.add(
                participant.name
                    + ": available="
                    + participant.available
                    + ", pending="
                    + participant.hasPending(txId)
                    + ", committed="
                    + participant.hasCommitted(txId)
            );
        }
        return lines;
    }
}
```